Mục lục

2 Phần cứng IoT (IoT Hardware) 1

2.1 Mục tiêu học tập và vị trí trong kiến trúc IoT 1
2.2 Khái niệm và đặc tính chuyển đổi 1
2.2.1 Các đại lượng ảnh hưởng chất lượng phép đo (mở rộng kỹ thuật) 2
2.3 Phân loại cảm biến 2
2.3.1 Cảm biến tích cực và thụ động 2
2.3.2 Tín hiệu tương tự và tín hiệu số 3
2.3.3 Phân loại theo đại lượng và cơ chế 3
2.4 Kết nối cảm biến với vi điều khiển (mở rộng kỹ thuật) 4
2.4.1 ADC, lượng tử hóa và ví dụ tính toán 4
2.4.2 Cảm biến điện trở: mạch phân áp 4
2.4.3 I^2C, SPI, UART và ngõ vào số 5
2.5 Mô hình nút cảm biến IoT 5
2.5.1 Nguồn và độ tin cậy đo 5
2.6 Ví dụ thiết kế: đo nhiệt độ và điều khiển quạt 5
2.7 Câu hỏi ôn tập và bài tập 6
2.8 Mục tiêu và khái niệm 6
2.9 So sánh cảm biến và cơ cấu chấp hành 6
2.10 Nhóm phần tử chấp hành thường gặp 7
2.10.1 LED, màn hình và còi báo 7
2.10.2 Động cơ DC, động cơ bước và servo 7
2.10.3 Rơ-le, van điện từ, tải nhiệt và cơ cấu tuyến tính 8
2.11 Nguyên lý ghép mạch điều khiển công suất 8
2.11.1 Vì sao GPIO không cấp trực tiếp được tải lớn? 8
2.11.2 PWM: điều chỉnh công suất trung bình 8
2.12 Đặc tính kỹ thuật khi chọn actuator 9
2.13 Điều khiển vòng hở và vòng kín 9
2.14 Ví dụ chương trình Arduino: LED bằng PWM 9
2.15 Bài tập và câu hỏi ôn tập 10
2.16 Kiến trúc hệ thống nhúng và mục tiêu học tập 10
2.17 ATmega328P: nền tảng AVR 8 bit 10
2.17.1 Flash, SRAM, EEPROM: ba vai trò khác nhau 11
2.17.2 Chân nguồn, I/O và nạp chương trình 11
2.18 Arduino: hệ sinh thái phần cứng/phần mềm mở 12

2.18.1 Các bo Arduino trong bài giảng . . . 12
2.18.2 Phân tích một bo Uno cổ điển . . . 12
2.18.3 Shield, breadboard, kit và các nền tảng thay thế . . . 13
2.19 Lập trình Arduino IDE và ngôn ngữ Sketch . . . 13
2.19.1 Chu trình công cụ và chương trình . . . 13
2.19.2 Hai hàm `setup()` và `loop()` . . . 13
2.19.3 Lệnh và khái niệm quan trọng . . . 14
2.19.4 Ví dụ: đọc analog và gửi Serial . . . 14
2.19.5 Serial UART và COM . . . 15
2.20 ESP8266 và các module phổ biến . . . 15
2.20.1 ESP-01, ESP-07 và NodeMCU . . . 15
2.20.2 Firmware, SDK và quy trình IDE . . . 15
2.21 ESP32: một họ SoC, không phải một chip duy nhất . . . 15
2.21.1 Lợi thế và giới hạn thực dụng . . . 16
2.21.2 So sánh các lựa chọn MCU căn bản . . . 16
2.22 Intel Edison: ví dụ nền tảng lai có tính lịch sử . . . 16
2.23 Edge Computing trong kiến trúc IoT . . . 17
2.23.1 So sánh các vị trí xử lý . . . 17
2.24 Raspberry Pi: máy tính đơn bo cho gateway và Edge . . . 17
2.24.1 Các thành phần và GPIO . . . 18
2.24.2 Dựng gateway cơ bản với Raspberry Pi . . . 18
2.24.3 Arduino và Raspberry Pi: không có nền tảng nào tốt nhất cho mọi việc . . . 18
2.25 Thiết kế nút IoT: bài toán tổng hợp nhiều nền tảng . . . 19
2.25.1 Bảng quyết định chọn phần cứng . . . 19
2.25.2 Công thức sơ bộ về lưu trữ và truyền thông . . . 19
2.25.3 Ví dụ hệ IoT đầy đủ: giám sát phòng học . . . 19
2.26 Câu hỏi tự luyện và đáp án ngắn . . . 20
2.27 Bảng thuật ngữ ôn thi . . . 20

CHƯƠNG 2
Phần cứng IoT (IoT Hardware)

Ghi nhớ

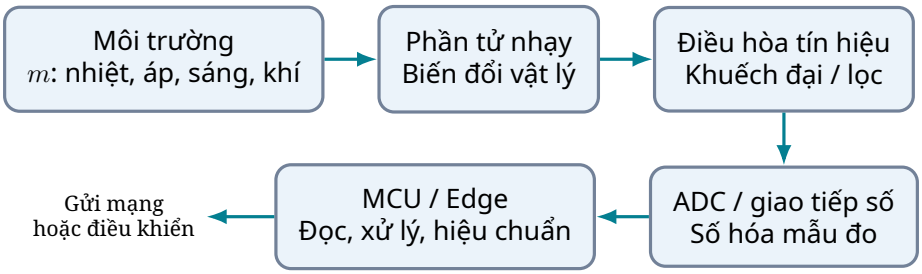
Phạm vi: Cảm biến, cơ cấu chấp hành và vi điều khiển thuộc **Chương 2 – IoT Hardware** của TS. Nguyễn Đắc Cử.

2.1 Mục tiêu học tập và vị trí trong kiến trúc IoT

Hoàn thành chương này, người học cần: định nghĩa được cảm biến; mô tả đường đi của đại lượng đo đến dữ liệu số; phân biệt cảm biến tích cực/thụ động và đầu ra tương tự/số; nhận diện các cảm biến thường gặp; tính toán giá trị ADC và lựa chọn giao tiếp phù hợp; thiết kế sơ bộ nút thu thập dữ liệu với chú ý về nguồn, sai số và hiệu chuẩn.

Theo slide bài giảng, kiến trúc IoT được hình dung qua ba tầng gần gũi: **Things / IoT Devices** thu thập dữ liệu vật lý; **Edge** xử lý gần nơi phát sinh dữ liệu; và **Cloud** lưu trữ, phân tích, quản lý thông qua kết nối mạng. Phần cứng nút cảm biến là cầu nối giữa thế giới vật lý và luồng thông tin số.

Theo Chapter 2: IoT Hardware, slide PDF 3–4, 16–17.



Hình 2.1: Chuỗi thu thập dữ liệu cảm biến, sơ đồ biên soạn mới.

2.2 Khái niệm và đặc tính chuyển đổi

Cảm biến (*sensor*) là phần tử hoặc thiết bị nhận một đại lượng vật lý, hóa học, sinh học và tạo ra tín hiệu đo có thể đọc, xử lý hoặc truyền đi. Đại lượng được đo ký hiệu m (*measurand*); tín hiệu đầu ra là s; quan hệ chuyển đổi được viết

$$\boxed{s = f(m).} \tag{2.1}$$

Trường hợp gần tuyến tính trong một miền làm việc: $s = km + b$. Hệ số $k = \Delta s/\Delta m$ được gọi là *độ nhạy* nếu các điều kiện đo phù hợp. Tín hiệu có thể là điện áp, dòng điện, điện trở, điện dung, tần số, xung hoặc chuỗi bit qua bus số. Trên thực tế, nhiều cảm biến cần *mạch kích thích* và mạch điều hòa để tạo ra đầu ra phù hợp với bộ xử lý.

Theo Chapter 2: IoT Hardware, slide PDF 4.

2.2.1 Các đại lượng ảnh hưởng chất lượng phép đo (mở rộng kỹ thuật)

Khoảng đo (range)
Khoảng đại lượng mà thiết bị cho phép đo theo công bố của nhà sản xuất; vượt khoảng đo có thể bão hòa hoặc hư hỏng.

Độ phân giải (resolution)
Thay đổi nhỏ nhất có thể phân biệt tại đầu ra; độ phân giải số không đồng nghĩa với độ chính xác.

Độ chính xác (accuracy)
Mức độ gần giá trị đo với giá trị chuẩn; thường được nêu theo $\pm$ đơn vị hoặc phần trăm.

Độ lặp lại (repeatability) và độ chính xác lặp (precision)
Mức phân tán kết quả trong điều kiện đo giống nhau; kết quả lặp tốt vẫn có thể lệch chuẩn.

Độ nhạy, độ tuyến tính, độ trễ
Mô tả mức đáp ứng, sai lệch so với mô hình tuyến tính và sự phụ thuộc vào chiều tăng/giảm đại lượng.

Thời gian đáp ứng, dải thông
Giới hạn tốc độ phản ứng, ảnh hưởng trực tiếp đến tần số lấy mẫu phù hợp.

Độ trôi (drift)
Biến đổi điểm không hoặc độ nhạy do nhiệt độ, lão hóa và môi trường. Phải xem xét hiệu chuẩn định kỳ.

2.3 Phân loại cảm biến

Slide cung cấp ba hướng phân loại: theo *nguồn năng lượng/tác động kích thích*, theo *đại lượng hay hiện tượng chuyển đổi* và theo *dạng đầu ra*. Không nên hiểu đây là ba cách phân loại loại trừ nhau: một cảm biến nhiệt điện trở có thể đồng thời được gọi là cảm biến nhiệt, cảm biến điện trở và cảm biến đầu ra analog khi ghép với mạch đọc.

Theo Chapter 2: IoT Hardware, slide PDF 7–12.

2.3.1 Cảm biến tích cực và thụ động

Bài giảng mô tả cảm biến **tích cực** (*active*) là cảm biến cần năng lượng kích thích bên ngoài, ví dụ radar/sonar chủ động phát sóng và đo phản xạ; cảm biến **thụ động** (*passive*) là cảm biến không cần chủ động phát tín hiệu đo, ví dụ cặp nhiệt điện tự phát sinh sức điện động từ chênh lệch nhiệt độ.

Ghi nhớ

Phân biệt quan trọng: Trong tài liệu chuyên ngành, “active/passive” không phải lúc nào cũng được định nghĩa thống nhất. Việc cảm biến *không chủ động phát sóng* khác với việc *không cần nguồn cấp*. Camera thụ động thu ánh sáng môi trường nhưng vẫn

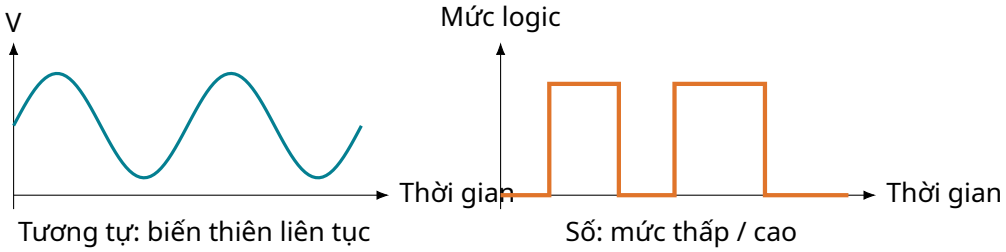
cần điện để hoạt động. Một cảm biến áp suất kiểu điện trở/điện dung thường cần mạch kích thích, trong khi phần tử áp điện có thể tự tạo điện tích. Khi trả lời câu hỏi, nêu rõ tiêu chí phân loại được sử dụng.

Bảng 2.1: Đối chiếu nguyên lý theo cách mô tả của bài giảng

Tiêu chí	Tích cực (phát tín hiệu dò)	Thụ động (khai thác tín hiệu sẵn có)
Nguyên lý	Phát siêu âm, vô tuyến hoặc ánh sáng rồi đo đáp ứng	Thu tín hiệu từ đại lượng tự nhiên hoặc chuyển đổi trực tiếp
Ví dụ	Radar đo khoảng cách; siêu âm HC-SR04	Cặp nhiệt điện; quang trở nhận ánh sáng
Ưu điểm	Chủ động phép đo, ít phụ thuộc một số nguồn kích thích bên ngoài	Có thể đơn giản, ít ảnh hưởng tới đối tượng
Lưu ý	Công suất phát, phản xạ và nhiễu đồng kênh	Không đồng nghĩa toàn bộ thiết bị không dùng điện

2.3.2 Tín hiệu tương tự và tín hiệu số

Tín hiệu tương tự (*analog*) thay đổi liên tục theo thời gian/giá trị (trong mô hình lý tưởng). **Tín hiệu số** (*digital*) biểu diễn thông tin bằng các mức rời rạc; tín hiệu logic vật lý thường biểu diễn mức LOW/HIGH (0/1), còn dữ liệu đo số có thể mã hóa hàng chục bit và nhiều giá trị, không bị giới hạn ở hai giá trị của đại lượng đo.



Hình 2.2: Hai dạng tín hiệu thường gặp ở đầu vào vi điều khiển.

Một *cảm biến analog* có thể xuất điện áp tỷ lệ đại lượng đo (hoặc thay đổi điện trở cần mạch phân áp). Bộ điều khiển cần ADC để chuyển thành mã số. Một *cảm biến số* tích hợp ADC, xử lý/hiệu chuẩn hoặc bộ giao tiếp số, truyền kết quả qua I^2C, SPI, UART, 1-Wire; cũng có thiết bị chỉ tạo xung logic. Do đó nhận định trong slide rằng cảm biến số "có dải đo thấp" không phải tính chất phổ quát: phải đối chiếu thông số của *từng model*.

Theo Chapter 2: IoT Hardware, slide PDF 9–12.

2.3.3 Phân loại theo đại lượng và cơ chế

Ví dụ cảm biến trong slide gồm: nhiệt độ, ẩm đất, độ ẩm không khí, hồng ngoại, khí gas, CO_2, nút nhấn, alcohol, siêu âm, LDR, rain, photodiode, proximity, PIR, gyroscope. Cần phân biệt *nguyên lý* và *board/module*: cùng là cảm biến nhiệt độ nhưng có module analog, digital và cảm biến tiếp xúc/không tiếp xúc khác nhau.

Theo Chapter 2: IoT Hardware, slide PDF 5–6, 11–12.

Bảng 2.2: Các cảm biến minh họa trong bài giảng và công dụng

Nhóm	Ví dụ trong slide	Đại lượng / ứng dụng
Nhiệt độ	Temperature sensor; cặp nhiệt điện	Giám sát nhiệt độ, bảo vệ thiết bị
Ẩm	Humidity, soil moisture	Độ ẩm không khí hoặc độ ẩm đất; không dùng lẫn hai đơn vị đo
Quang	LDR, photodiode, IR optical	Cường độ ánh sáng, phát hiện vật/đường line
Khoảng cách	Ultrasonic, IR proximity	Khoảng cách, nhận diện chướng ngại
Chuyển động	Accelerometer, gyroscope, PIR	Gia tốc, tốc độ góc, phát hiện chuyển động hồng ngoại
Môi trường	Gas, CO_2, rain, alcohol	Chỉ thị khí hoặc lượng mưa; phân biệt cảm biến khí tổng hợp và CO_2 chuyên dụng
Tương tác	Digital push button	Trạng thái nhấn/thả, thường là tín hiệu logic
Áp suất	Pressure sensor	Áp suất khí/quá trình, suy ra cao độ trong điều kiện phù hợp

2.4 Kết nối cảm biến với vi điều khiển (mở rộng kỹ thuật)

2.4.1 ADC, lượng tử hóa và ví dụ tính toán

ADC N bit tạo ra 2^N mức mã, thông thường từ 0 đến $2^N - 1$. Với ADC đơn cực lý tưởng tham chiếu V_{ref} và điện áp hợp lệ trong $[0, V_{\text{ref}}]$, công thức quy đổi xấp xỉ từ mã D là

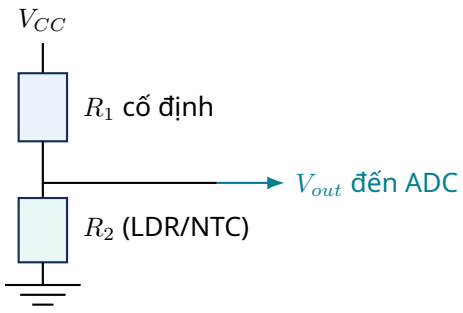
$$\boxed{V_{\text{in}} \approx \frac{D}{2^N - 1} V_{\text{ref}}} \qquad (0 \le D \le 2^N - 1). \tag{2.2}$$

Bước lượng tử hóa danh định thường viết $\Delta V \approx V_{\text{ref}}/2^N$; một số cách quy đổi cuối thang dùng $2^N - 1$. Phải tuân theo quy ước của ADC và thư viện cụ thể. Ví dụ ADC 10 bit, $V_{\text{ref}} = 5\,\text{V}$, nhận $D = 512$: $V_{\text{in}} \approx 512 \times 5/1023 \approx 2.50\,\text{V}$. Đây là **độ phân giải mã**, không nói trước sai số hệ thống.

2.4.2 Cảm biến điện trở: mạch phân áp

Một phần tử cảm biến thay đổi điện trở, chẳng hạn LDR hay NTC, cần được đo dưới dạng điện áp. Với R_1 nối từ nguồn V_{CC} đến nút đo, R_2 nối từ nút đo xuống GND:

$$V_{\text{out}} = V_{CC} \frac{R_2}{R_1 + R_2}. \tag{2.3}$$



Hình 2.3: Đấu nối phân áp nguyên lý. Kiểm tra điện áp nguồn trước khi đưa vào ADC.

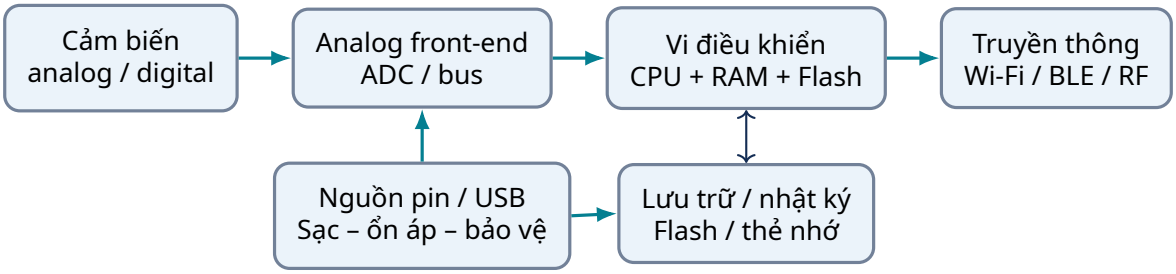
Bảng 2.3: Khi nào dùng giao tiếp nào?

Giao tiếp	Dây và nguyên lý	Lưu ý thiết kế
GPIO số	Một chân đọc HIGH/LOW, ngắt cạnh	Cần điện trở kéo lên/kéo xuống phù hợp; chống dội nút nhấn
ADC	Một chân analog tham chiếu mức điện áp	Chống aliasing, lọc nhiễu, hiệu chuẩn, không quá áp
I^2C	SDA/SCL, địa chỉ thiết bị trên bus	Điện trở kéo lên, địa chỉ trùng, điện áp logic, chiều dài bus
SPI	SCLK, MOSI, MISO, CS	Nhanh, nhiều dây; mỗi slave thường cần CS
UART	TX/RX, mức baud, định dạng bit	Nối chéo TX-RX, GND chung, không nhầm TTL với RS-232
1-Wire	Dây dữ liệu cùng GND, giao thức định thời	Điện trở kéo lên; thời gian và chiều dài đường truyền

2.4.3 I^2C, SPI, UART và ngõ vào số

2.5 Mô hình nút cảm biến IoT

Theo sơ đồ trang 16–17 slide, nút IoT gồm *Sensors* + *Board* + *Connectivity* và được phân rã thành cảm biến, mạch tiền xử lý/ADC, vi điều khiển, bộ nhớ, truyền thông có dây/không dây và khối nguồn. Lớp ứng dụng còn cần firmware, định danh và bảo mật.



Hình 2.4: Sơ đồ khối cấu trúc nút IoT tái dựng từ nội dung bài giảng.

Theo Chapter 2: IoT Hardware, slide PDF 16–17.

2.5.1 Nguồn và độ tin cậy đo

Thiết kế nút cảm biến cần lưu ý dòng khởi động và dòng đỉnh của RF, mức nghỉ của cảm biến, chu kỳ đo, khả năng giảm công suất khi ngủ và tính toàn vẹn dữ liệu. Phương pháp ước lượng cơ bản:

$$I_{\text{avg}} = \sum_i D_i I_i, \qquad T_{\text{h}} \approx \frac{C_{\text{usable}}\,(\text{mAh})}{I_{\text{avg}}\,(\text{mA})}. \tag{2.4}$$

Đây là ước tính lý tưởng theo dung lượng *khả dụng*, chưa tính hiệu suất DC/DC, tự xả, suy giảm pin và nhiệt độ. Chu kỳ lấy mẫu nên dựa trên dải tần tín hiệu, yêu cầu trễ, tuổi thọ pin và chi phí truyền thông; lấy mẫu nhanh hơn không mặc nhiên làm dữ liệu chính xác hơn.

2.6 Ví dụ thiết kế: đo nhiệt độ và điều khiển quạt

Một cảm biến đọc nhiệt độ theo chu kỳ, MCU lọc trung bình trượt, so sánh với hai ngưỡng và xuất tín hiệu đến mạch công suất điều khiển quạt. Hai ngưỡng $T_{\text{on}} > T_{\text{off}}$ tạo *hysteresis*,

tránh đóng ngắt liên tục. Nếu sensor báo lỗi, hệ thống cần chính sách rõ ràng: tắt an toàn, báo lỗi hoặc bật quạt phòng ngừa tùy nguy cơ.

Ví dụ giả định: bật quạt khi $T \geq 32°\text{C}$, tắt khi $T \leq 29°\text{C}$. Mức này chỉ minh họa thuật toán, không phải nhiệt độ khuyến nghị cho một loại thiết bị cụ thể. MCU không được trực tiếp cấp dòng cho mô-tơ.

2.7 Câu hỏi ôn tập và bài tập

1. Trình bày $s = f(m)$, phân biệt tín hiệu đầu ra của cảm biến với đại lượng vật lý cần đo. Nêu một ví dụ có đường truyền xử lý hoàn chỉnh.
2. Theo phân loại của bài giảng, giải thích tại sao radar và sonar được coi là chủ động, cặp nhiệt điện là thụ động; nhận xét việc dùng từ "không cần năng lượng".
3. So sánh cảm biến analog với cảm biến digital; giải thích vì sao một cảm biến có giao tiếp I^2C có thể xuất nhiều giá trị đo dù tín hiệu trên dây là logic 0/1.
4. Tính mã ADC 10 bit gần đúng khi điện áp vào 1.65 V và $V_{ref} = 3.3$ V. *Đáp số:* gần 512.
5. Với phân áp $R_1 = 10\,\text{k}\Omega$, $R_2 = 5\,\text{k}\Omega$ và $V_{CC} = 3.3$ V, tính V_{out}. *Đáp số:* 1.1 V.
6. Vẽ lại nút IoT dùng cảm biến nhiệt độ số, ESP32, Wi-Fi, nguồn pin và cảnh báo LED; ghi bus, điện áp logic, hướng truyền dữ liệu.
7. Phân tích ảnh hưởng của nhiệt độ, nhiễu nguồn, vị trí lắp đặt, sai số hiệu chuẩn đến độ tin cậy phép đo.

Ghi nhớ

Tóm tắt cần nhớ: Cảm biến biến đổi đại lượng thực thành tín hiệu; hiểu đúng đơn vị, dải đo, sai số và điện áp đầu ra quan trọng hơn việc ghi nhớ tên module. Ba trục phân loại (kích thích, hiện tượng, dạng đầu ra) có thể chồng lấp. Một nút IoT hoàn chỉnh cần cảm biến, mạch ghép nối, xử lý, nguồn và truyền thông.

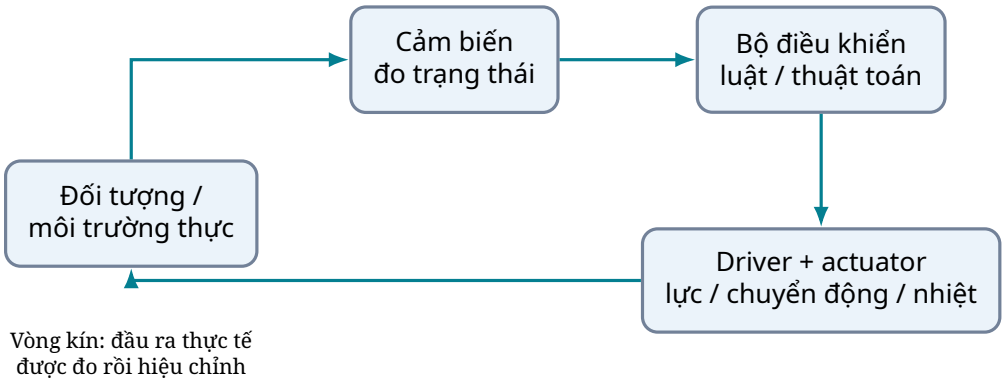
2.8 Mục tiêu và khái niệm

Sau chương này, người học cần mô tả được vai trò cơ cấu chấp hành trong IoT; phân biệt cảm biến/đầu vào với đầu ra vật lý; chọn sơ bộ phần tử chấp hành; hiểu nguyên lý GPIO, PWM và mạch driver; biết các ràng buộc nguồn, bảo vệ và an toàn khi ghép MCU với tải.

Theo slide, **cơ cấu chấp hành** (*actuator*) là thiết bị nhận tín hiệu điều khiển (thường là điện) và tạo ra chuyển động cơ học hoặc một thay đổi vật lý như ánh sáng, âm thanh, nhiệt độ, áp lực, trạng thái đóng mở. Các ví dụ minh họa bao gồm động cơ điện, động cơ bước, LED, LCD, ma trận LED, còi/buzzer. Trong ngôn ngữ hẹp, "actuator" có thể chỉ cơ cấu tạo chuyển động; ở phạm vi bài giảng IoT, từ này được dùng rộng hơn để chỉ *thiết bị đầu ra tác động lên môi trường hoặc người sử dụng*.

Theo Chapter 2: IoT Hardware, slide PDF 13–15.

2.9 So sánh cảm biến và cơ cấu chấp hành



Hình 2.5: Vòng điều khiển phản hồi: cảm biến là đường đo; actuator là đường tác động.

Bảng 2.4: Hai vai trò trong hệ IoT, tổng hợp từ slide 14

Thuộc tính	Cảm biến (sensor)	Chấp hành (actuator)
Vai trò	Đọc hoặc cảm nhận thế giới thực	Tác động vào thế giới thực
Hướng dữ liệu	Vật lý → tín hiệu → MCU	MCU → lệnh / driver → thay đổi vật lý
Vị trí chức năng	Nhánh đầu vào hệ điều khiển	Nhánh đầu ra hệ điều khiển
Ví dụ	Nhiệt kế, micro, IMU, từ kế	Quạt, LED, loa, van điện từ, servo
Sai lầm hay gặp	Nhầm mức logic sensor là tải	Nối động cơ trực tiếp với chân GPIO

2.10 Nhóm phần tử chấp hành thường gặp

2.10.1 LED, màn hình và còi báo

LED biến điện năng thành ánh sáng, phù hợp báo trạng thái. LED đơn cần điện trở hạn dòng $R \approx (V_{CC} - V_F)/I$ khi điện áp cấp lớn hơn điện áp thuận V_F và phù hợp với khả năng dòng chân MCU. Chẳng hạn với $V_{CC} = 3.3$ V, $V_F = 2.0$ V, mục tiêu $I = 5$ mA: $R \approx 260\,\Omega$, có thể chọn điện trở chuẩn gần đó và tính lại dòng. Màn hình LCD, ma trận LED cần nhiều tín hiệu dữ liệu và mạch driver; buzzer có loại *active* tự tạo âm khi cấp điện và *passive* cần tín hiệu dao động. Tên “active/passive” của buzzer không đồng nghĩa cách phân loại cảm biến.

2.10.2 Động cơ DC, động cơ bước và servo

Động cơ DC: tốc độ xấp xỉ điều khiển qua điện áp hiệu dụng/PWM; chiều quay có thể đảo bằng cầu H. Dòng khởi động/khóa trục có thể lớn hơn nhiều so với dòng chạy không tải, do đó cần driver và nguồn được tính theo tải thực.

Động cơ bước (stepper): quay theo từng bước rời rạc khi driver được cấp chuỗi xung pha. Với động cơ có góc bước θ_{step}, nếu điều khiển bước đầy đủ và không bị mất bước: $N_{rev} = 360^\circ/\theta_{step}$. Ví dụ động cơ 1.8°/bước cần 200 bước/vòng. Vi bước (microstepping) tăng số vi bước lệnh trên một vòng nhưng không bảo đảm độ chính xác cơ khí tương ứng. Slide sử dụng stepper để minh họa điều khiển vị trí trong cánh tay robot.

Servo RC: thường có bộ điều khiển bên trong, nhận xung điều khiển theo chu kỳ; góc, dải xung và tốc độ tùy servo. Cần nguồn đủ dòng, GND chung và không mặc định rằng mọi servo hỗ trợ cùng phạm vi góc.

Theo Chapter 2: IoT Hardware, slide PDF 13.

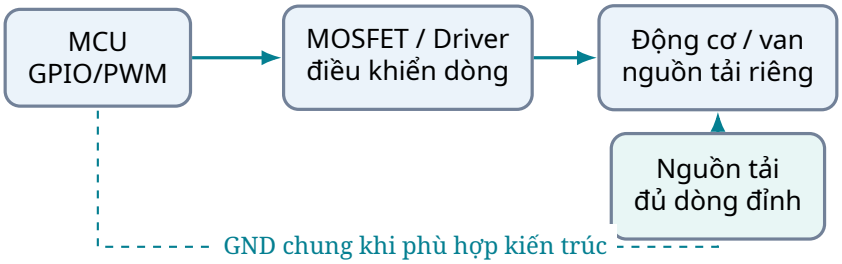
2.10.3 Rơ-le, van điện từ, tải nhiệt và cơ cấu tuyến tính

Rơ-le dùng cuộn hút để thay đổi trạng thái tiếp điểm và cách ly chức năng điều khiển khỏi tải, nhưng phải đáp ứng định mức điện áp/dòng, khoảng cách điện và chuẩn an toàn. Van điện từ dùng từ trường điều khiển dòng lưu chất; động cơ tuyến tính/solenoid dùng lực điện từ để kéo hoặc đẩy. Dây đốt nóng và module Peltier biến điện thành nhiệt/lạnh tương đối; cần cảm biến phản hồi, bảo vệ quá nhiệt và nguồn công suất phù hợp.

2.11 Nguyên lý ghép mạch điều khiển công suất

2.11.1 Vì sao GPIO không cấp trực tiếp được tải lớn?

GPIO xuất mức logic và dòng giới hạn bởi datasheet MCU; mô-tơ, van, rơ-le tiêu thụ dòng lớn và có thể phát xung quá áp khi ngắt. Mạch driver (MOSFET, BJT, cầu H, IC chuyên dụng) chuyển tín hiệu mức thấp thành chuyển mạch tải. Với tải cuộn cảm DC, diode *flyback* mắc ngược chiều dẫn bình thường tạo đường cho dòng cảm khi ngắt. Tải AC/công suất lớn cần mạch và biện pháp an toàn riêng, không sử dụng sơ đồ DC đơn giản một cách máy móc.



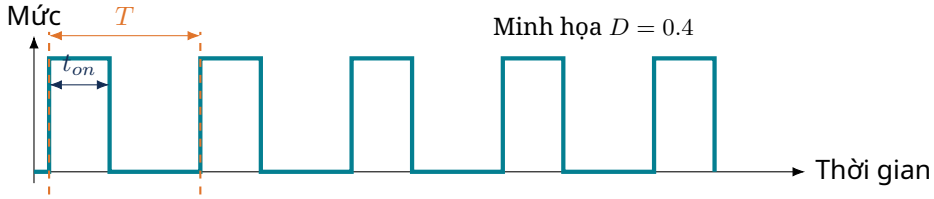
Hình 2.6: Phân tách tín hiệu điều khiển và đường công suất.

2.11.2 PWM: điều chỉnh công suất trung bình

PWM (*pulse-width modulation*) đóng/ngắt tải theo chu kỳ T với thời gian ON là t_{on}; hệ số lấp đầy:

$$\boxed{D = \frac{t_{on}}{T}}, \qquad 0 \le D \le 1, \qquad f_{PWM} = \frac{1}{T}. \tag{2.5}$$

Với tải có tính quán tính/lọc đủ lớn, công suất hoặc điện áp *trung bình* có thể phụ thuộc D. Ví dụ PWM mức cao 3.3 V, $D = 0.40$ cho điện áp trung bình lý tưởng khoảng 1.32 V *tại tín hiệu PWM*, nhưng không nên coi đó là điện áp DC sạch và không suy ra ngay mô-men động cơ. Tần số PWM chọn theo driver, tiếng ồn, hiệu suất và yêu cầu điều khiển.



Hình 2.7: Xung PWM 40% duty cycle (sơ đồ lý tưởng).

2.12 Đặc tính kỹ thuật khi chọn actuator

Bảng 2.5: Bảng kiểm tham số thiết kế

Tham số	Câu hỏi cần trả lời	Rủi ro nếu bỏ qua
Công suất / dòng	Điện áp danh định? Dòng chạy, dòng đỉnh, dòng khởi động?	Sụt áp, reset MCU, quá nhiệt
Driver	Cần MOSFET, relay, cầu H, IC điều khiển bước?	Hỏng GPIO, không đảo chiều được
Tốc độ / vị trí	Tốc độ, hành trình, mô-men và độ chính xác cần bao nhiêu?	Mất bước, quá tải cơ khí
Điều khiển	On/off, PWM, xung bước, UART, I^2C?	Không tương thích MCU/driver
Phản hồi	Có encoder, công tắc hành trình, cảm biến dòng?	Không phát hiện kẹt tải
Điện từ / EMC	Tải cảm có diode dập xung, tụ, snubber phù hợp?	Reset, nhiễu sensor/RF
An toàn	Khi lỗi mất mạng hoặc MCU treo, tải về trạng thái nào?	Tải hoạt động không kiểm soát
Cơ khí	Giới hạn nhiệt, lắp gá, rung, chống nước?	Mòn, hỏng, mất ổn định

2.13 Điều khiển vòng hở và vòng kín

Vòng hở (*open-loop*) phát lệnh mà không đo lại kết quả. Ví dụ cấp 100 xung bước và giả định trục quay đúng 100 bước; mô hình đơn giản, chi phí thấp nhưng có thể sai lệch do kẹt tải. **Vòng kín** (*closed-loop*) thu phản hồi để tính sai lệch $e(t) = r(t) - y(t)$, sau đó điều chỉnh lệnh $u(t)$. Các thuật toán thường gặp gồm on/off có hysteresis, PID và điều khiển trạng thái.

Ví dụ nhà thông minh: đặt nhiệt độ mục tiêu, sensor đo không khí, MCU xử lý, relay/MOSFET điều khiển quạt. Với cơ cấu chấp hành động cơ, nếu cần bảo đảm tốc độ thực cần encoder hoặc cảm biến phản hồi thích hợp.

2.14 Ví dụ chương trình Arduino: LED bằng PWM

Đoạn mã sau minh họa sử dụng bo Arduino Uno cổ điển với một chân PWM phù hợp. Chọn điện trở hạn dòng LED theo phần cứng thực tế; `analogWrite()` tạo PWM ở nhiều bo Arduino, *không* phải DAC analog thật.

Listing 2.1: Tăng/giảm độ sáng LED theo dạng tam giác

```
const int LED_PIN = 9;
void setup() {
  pinMode(LED_PIN, OUTPUT);
}
void loop() {
  for (int duty = 0; duty <= 255; duty += 5) {
    analogWrite(LED_PIN, duty);
    delay(25);
  }
```

```
  for (int duty = 255; duty >= 0; duty -= 5) {
    analogWrite(LED_PIN, duty);
    delay(25);
  }
}
```

Mã chỉ điều khiển LED công suất nhỏ trong giới hạn cho phép; muốn điều khiển bơm/van/động cơ phải thêm driver, cấp nguồn và bảo vệ.

2.15 Bài tập và câu hỏi ôn tập

1. Nêu ít nhất bốn khác biệt giữa cảm biến và cơ cấu chấp hành theo luồng năng lượng, thông tin và chức năng.
2. Tại sao động cơ bước điều khiển vị trí chính xác hơn DC motor vòng hở trong nhiều tình huống, và khi nào vẫn cần encoder?
3. Một động cơ có góc bước $1.8°$; muốn quay $90°$ ở chế độ bước đầy đủ, cần bao nhiêu bước? *Đáp số:* 50 bước.
4. PWM có $T = 10$ ms và thời gian ON 3.5 ms. Tính duty, tần số. *Đáp số:* 35%, 100 Hz.
5. Vẽ mạch khối ESP32 $\rightarrow$ driver MOSFET $\rightarrow$ bơm DC, xác định nguồn tải, GND, bảo vệ diode và tín hiệu điều khiển.
6. Nêu cách xử lý nếu sensor mất kết nối khi actuator đang bật. Vì sao không có chính sách lỗi chung cho mọi thiết bị?

Ghi nhớ

Tóm tắt cần nhớ: Actuator tạo tác động vật lý từ tín hiệu điều khiển. GPIO và driver là hai tầng khác nhau. Cần chọn nguồn theo dòng đỉnh, thiết kế bảo vệ tải cảm, phản hồi và trạng thái an toàn. PWM là điều chế độ rộng xung; không phải điện áp analog thuần.

2.16 Kiến trúc hệ thống nhúng và mục tiêu học tập

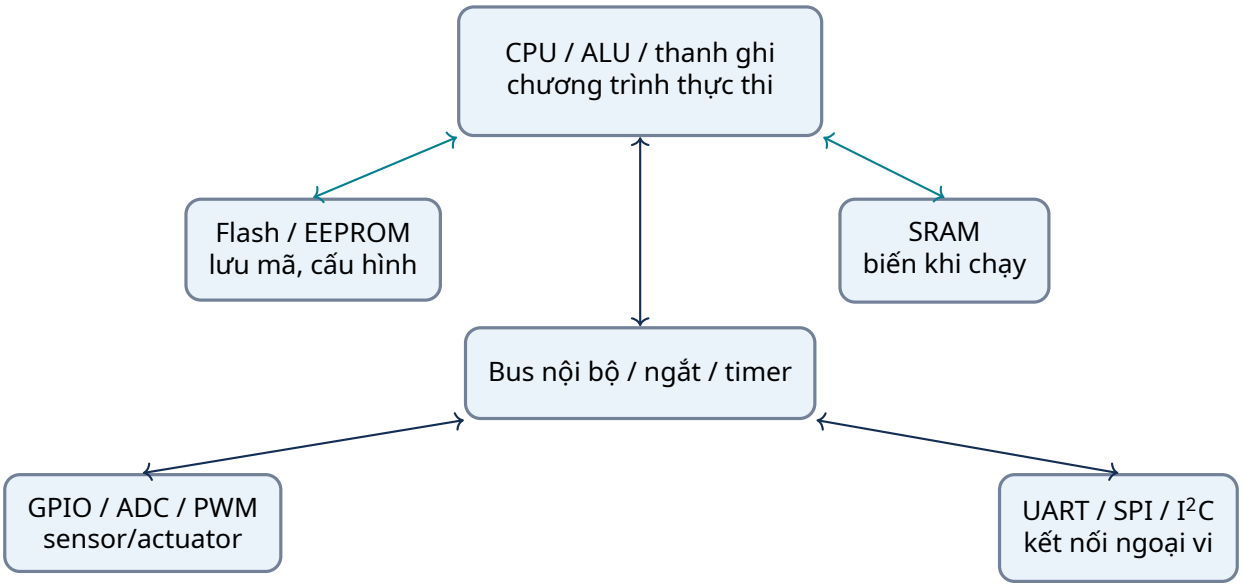
Sau chương này, người học cần hiểu sự khác nhau giữa chip vi điều khiển, bo phát triển và máy tính đơn bo; biết cấu trúc CPU–Flash–RAM–I/O; mô tả Arduino Uno, Leonardo, Mega, Nano, ATmega328P, ESP8266, ESP32, Intel Edison, Raspberry Pi; hiểu thao tác lập trình Arduino IDE, nối cảm biến, dùng UART và lựa chọn xử lý ở Edge hay Cloud.

Vi điều khiển (*microcontroller/MCU*) là vi mạch tích hợp bộ xử lý, bộ nhớ và ngoại vi vào một hệ lập trình được, tối ưu cho các tác vụ nhúng. Một **bo phát triển** (*development board*) đưa chip ra các chân tiện dụng và có thêm mạch nguồn, USB, clock, bộ nạp. Một **máy tính đơn bo** (*single-board computer/SBC*) như Raspberry Pi thường có SoC mạnh hơn, RAM ngoài, lưu trữ và khả năng chạy hệ điều hành đa nhiệm. Do đó **Arduino không đồng nghĩa với chip ATmega**, và **Raspberry Pi không nên được coi là MCU theo nghĩa hẹp**.

Theo Chapter 2: IoT Hardware, slide PDF 18, 22–26, 73–78.

2.17 ATmega328P: nền tảng AVR 8 bit

Slide trình bày ATmega328P là MCU AVR 8-bit hiệu năng tốt, công suất thấp, bộ nhớ Flash 32 KB, SRAM 2 KB, EEPROM 1 KB, tối đa 23 đường I/O lập trình được trên chip, dải điện áp



Hình 2.8: Kiến trúc MCU mức khái niệm (sơ đồ mới, không phải sơ đồ silicon cụ thể).

hoạt động tùy tần số/phiên bản 1.8–$5.5\,\mathrm{V}$, vùng nhiệt độ công bố trong slide -40 đến $85\,^{\circ}\mathrm{C}$. Các giá trị dòng tiết kiệm năng lượng đưa ra trong slide (active $0.2\,\mathrm{mA}$, power-down $0.1\,\mu\mathrm{A}$, power-save $0.75\,\mu\mathrm{A}$) *chỉ là số liệu dưới điều kiện kiểm thử riêng*, không thể mặc nhiên dùng cho bo Arduino Uno khi có LED, bộ ổn áp và USB-UART.

Theo Chapter 2: IoT Hardware, slide PDF 19–21.

2.17.1 Flash, SRAM, EEPROM: ba vai trò khác nhau

Bảng 2.6: Bộ nhớ của ATmega328P theo bài giảng

Loại	Dung lượng	Chức năng
Flash	32 KB	Chứa chương trình và dữ liệu tĩnh, giữ khi mất nguồn
SRAM	2 KB	Biến lúc chạy, stack/heap; mất khi tắt nguồn
EEPROM	1 KB	Lưu cấu hình nhỏ khi mất nguồn; có giới hạn số chu kỳ ghi

Nếu khai báo quá nhiều biến động (mảng lớn, chuỗi, bộ đệm), hệ thống có thể lỗi do SRAM cạn dù Flash còn chỗ. Với dữ liệu ghi lặp nhiều, phải quan tâm mài mòn EEPROM và chiến lược ghi tối thiểu.

2.17.2 Chân nguồn, I/O và nạp chương trình

ATmega328P ở gói PDIP 28 chân thường có VCC, AVCC, GND, AREF, RESET, XTAL1/XTAL2 và các nhóm PORTB/C/D. Một chân vật lý có thể kiêm nhiều chức năng (GPIO, ADC, UART, SPI, PWM, ngắt), do đó *số chân package* không phải số GPIO thực dùng độc lập. Trong slide, giao tiếp nạp chương trình minh họa các tín hiệu RESET, SCK, MOSI, MISO, VCC, GND theo chuẩn ISP. Bootloader Arduino giúp nạp sketch qua USB-to-serial trên bo, trong khi ISP cho phép lập trình trực tiếp khi cần.

Theo Chapter 2: IoT Hardware, slide PDF 19, 21, 25, 27.

2.18 Arduino: hệ sinh thái phần cứng/phần mềm mở

Arduino là nền tảng điện tử mã nguồn mở gắn với bo và công cụ lập trình dễ tiếp cận, hỗ trợ nhiều hệ điều hành. Slide nhấn mạnh chi phí thấp, hỗ trợ đa nền tảng, môi trường lập trình đơn giản, phần mềm và phần cứng mở. Nội dung lịch sử trong slide nhắc dự án xuất phát từ môi trường giáo dục thiết kế tương tác tại Ivrea (Ý), phát triển thành hệ sinh thái bo, thư viện và shield rộng lớn.

Theo Chapter 2: IoT Hardware, slide PDF 22–24.

2.18.1 Các bo Arduino trong bài giảng

Bảng 2.7: Các bo điển hình theo slide; số liệu áp dụng cho phiên bản cổ điển được nêu

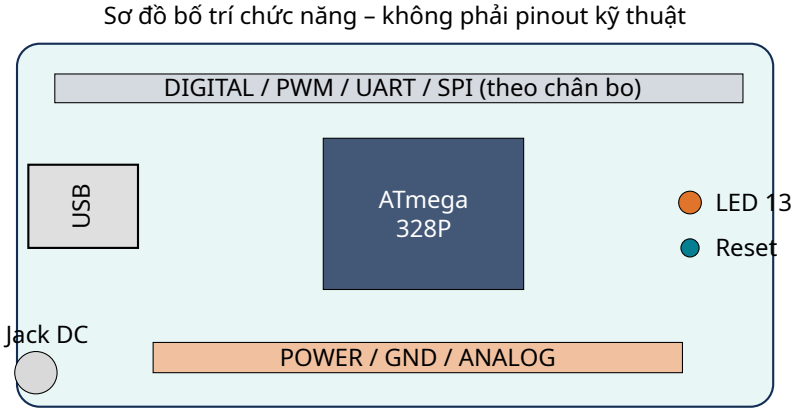
Bo	MCU	Flash / SRAM	Đầu vào/ra và ghi chú
Uno Rev3 (cổ điển)	ATmega328P, 8-bit	32 KB / 2 KB	14 chân digital, 6 analog; USB loại B; bootloader
Leonardo	ATmega32U4, 8-bit	32 KB / 2.5 KB	Tài liệu slide nêu 23 digital, 12 analog; USB tích hợp MCU
Mega 2560	ATmega2560, 8-bit	256 KB / 8 KB	54 digital, 16 analog; phù hợp khi cần nhiều chân
Nano (ATmega328)	ATmega328, 8-bit	32 KB / 2 KB	Thiết kế nhỏ; slide nêu 8 ngõ analog, 22 digital tổng hợp

Slide còn nhắc EEPROM 1 KB trên Uno/Nano/Leonardo, 4 KB trên Mega; dòng chân I/O và dải nguồn phải xét theo MCU, điện áp cấp *cho chip* khác với điện áp cho chân VIN hoặc jack bo mạch. Một số số lượng digital pin trong slide bao gồm chân đa chức năng, do vậy cần xem sơ đồ chân của *phiên bản bo* cụ thể. Arduino Uno mới (ví dụ R4) không dùng ATmega328P; không thể suy thông số từ bảng này cho mọi sản phẩm tên “Uno”.

Theo Chapter 2: IoT Hardware, slide PDF 25–30.

2.18.2 Phân tích một bo Uno cổ điển

Sơ đồ trang 26 slide chỉ ra các khu vực: hàng chân Digital, LED chân 13, LED báo nguồn, ATmega chính, hàng Analog, chân GND/5V, jack nguồn, đèn TX/RX, cổng USB và nút Reset. Chân Arduino D0/D1 thường gắn với UART RX/TX, nên dùng đồng thời với module UART và Serial Monitor có thể gây tranh chấp.



Hình 2.9: Sơ đồ khối bố trí thành phần tiêu biểu của Arduino Uno đời ATmega328P.

2.18.3 Shield, breadboard, kit và các nền tảng thay thế

Shield (*bo mở rộng*) cắm vào một số bo Arduino qua hàng chân tiêu chuẩn để bổ sung Ethernet, relay, màn hình, driver mô-tơ hoặc kết nối RF. Cần kiểm tra xung đột chân và tương thích mức logic. **Breadboard** (*bảng cắm thử không hàn*) thích hợp thử nghiệm nguyên mẫu, nhưng không lý tưởng cho dòng lớn, điện áp cao, tần số RF và rung động. Arduino kit thường gom các linh kiện, dây, sensor, servo và tài liệu thực hành.

Các phương án được slide giới thiệu gồm LaunchPad MSP430, Particle Photon, NodeMCU ESP8266, SparkFun Thing Plus, Feather HUZZAH, PocketBeagle, Seeeduino Nano, RedBoard Artemis và STM32F3 Discovery. Mỗi nền tảng có hệ sinh thái, CPU, mức logic và công cụ nạp khác nhau; được liệt kê để so sánh *khả năng lựa chọn*, không phải toàn bộ tương thích chân và mã Arduino.

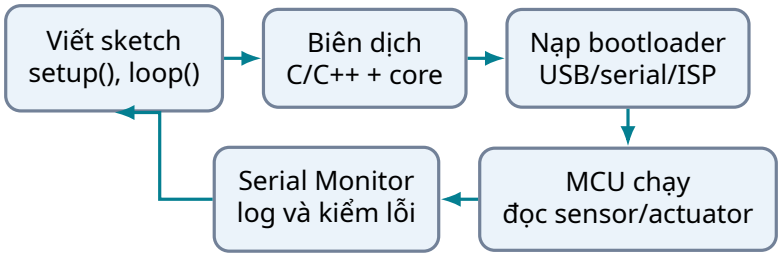
Theo Chapter 2: IoT Hardware, slide PDF 31–37.

2.19 Lập trình Arduino IDE và ngôn ngữ Sketch

2.19.1 Chu trình công cụ và chương trình

Slide gọi chương trình tải lên Arduino là **sketch** (*bản phác chương trình*); ngôn ngữ thực tế dựa trên C/C++ và các hàm của Arduino core. IDE cho phép lựa chọn board, cổng serial, quản lý thư viện, biên dịch, nạp và quan sát Serial Monitor. Quá trình cài board bên thứ ba (như ESP8266) thường đi qua Boards Manager; thư viện cài qua Library Manager hoặc các hình thức khác tùy dự án.

Theo Chapter 2: IoT Hardware, slide PDF 38–41, 51–52.



Hình 2.10: Chu trình thiết kế – nạp – kiểm thử firmware Arduino.

2.19.2 Hai hàm `setup()` và `loop()`

`setup()` chạy một lần sau khi khởi động hoặc reset để cấu hình chân, serial, thư viện và trạng thái ban đầu. `loop()` chạy lặp lại; muốn hệ thống phản ứng nhanh với nhiều công việc, nên ưu tiên lập lịch theo `millis()` thay vì nhiều lệnh `delay()` dài. Khi có lỗi hoặc yêu cầu dừng vĩnh viễn cần xử lý có chủ đích, tránh khối vòng lặp vô hạn vô tình.

Theo Chapter 2: IoT Hardware, slide PDF 41–51.

Listing 2.2: Cấu trúc sketch tối thiểu

```
const int LED_PIN = LED_BUILTIN;
void setup() {
  pinMode(LED_PIN, OUTPUT);
  Serial.begin(9600);
}
void loop() {
  digitalWrite(LED_PIN, HIGH);
  delay(500);
```

```
  digitalWrite(LED_PIN, LOW);
  delay(500);
}
```

2.19.3 Lệnh và khái niệm quan trọng

Bảng 2.8: Tóm tắt cú pháp được giới thiệu trong slide 40–50

Lệnh / cấu trúc	Ý nghĩa và chú ý
`//`, `/*...*/`	Chú thích một dòng và nhiều dòng; trình biên dịch bỏ qua nội dung chú thích.
`#define`, `#include`	Định nghĩa macro và thêm phần khai báo thư viện/header; khác với khai báo biến.
`if/else`	Rẽ nhánh nếu điều kiện đúng/sai; điều kiện là biểu thức logic.
`for`, `while`	Lặp có đếm hoặc lặp khi điều kiện còn đúng; đề phòng vòng lặp vô hạn.
`pinMode(pin,mode)`	Đặt chế độ chân (`INPUT`, `OUTPUT`, `INPUT_PULLUP` khi hỗ trợ).
`digitalRead(pin)`	Đọc trạng thái logic chân số (HIGH/LOW).
`digitalWrite(pin,value)`	Ghi mức HIGH/LOW lên GPIO output.
`analogRead(pin)`	Đọc ADC trả mã số; độ rộng/mức tham chiếu tùy bo.
`analogWrite(pin,value)`	Trên Uno cổ điển tạo PWM ở các chân hỗ trợ; độ rộng giá trị mặc định thường 0–255.
`delay(ms)`	Dừng tác vụ một khoảng thời gian (không nên dùng tràn lan trong tác vụ thời gian thực).
`abs()`, `max()`, `min()`, `pow()`	Các hàm toán thường dùng; để ý kiểu dữ liệu và tràn số.
`Serial.begin(baud)`	Khởi tạo tốc độ serial; tốc độ phía giám sát phải khớp.
`Serial.print/println()`	Gửi chuỗi/số lên terminal để theo dõi/debug.

2.19.4 Ví dụ: đọc analog và gửi Serial

Listing 2.3: Đọc ADC của Arduino Uno ATmega328P

```
const int ADC_PIN = A0;
void setup() {
  Serial.begin(9600);
}
void loop() {
  int raw = analogRead(ADC_PIN); // Uno: nominal 0..1023
  float voltage = raw * (5.0f / 1023.0f);
  Serial.print("ADC="); Serial.print(raw);
  Serial.print(" V="); Serial.println(voltage, 3);
  delay(250);
}
```

Công thức trên *giả định tham chiếu ADC đúng 5.0 V* và nguồn nằm trong mức an toàn, chỉ dùng để minh họa. Cần đo/tham chiếu Vref và xử lý sai số nếu đòi hỏi phép đo chính xác.

2.19.5 Serial UART và COM

Serial port truyền dữ liệu nối tiếp. Trong máy tính, "COM" có thể là cổng vật lý hoặc cổng ảo sinh bởi USB-to-serial, Bluetooth serial hoặc driver khác. Slide ghi chân 0 của Uno là RX, chân 1 là TX. UART TTL là mức logic của chip, khác *điện áp chuẩn RS-232*; không được nối trực tiếp phần cứng RS-232 truyền thống vào GPIO TTL nếu không có transceiver thích hợp. Mức baud, định dạng khung, GND và dây TX/RX phải được kiểm tra.

Theo Chapter 2: IoT Hardware, slide PDF 52–53.

2.20 ESP8266 và các module phổ biến

ESP8266EX (*Wi-Fi SoC*) của Espressif là vi mạch giá thấp tích hợp Wi-Fi và stack TCP/IP, có thể chạy ứng dụng độc lập hoặc đóng vai trò giao tiếp cho vi điều khiển khác. Slide đề cập TCP, UDP, HTTP, mDNS, DNS, OTA, Flash/SD, SPI, I^2C và hệ Arduino core; việc hỗ trợ từng tính năng phụ thuộc firmware, core, thư viện và chân được đưa ra module. ESP8266 chủ yếu hoạt động ở mức logic 3.3 V: không cấp trực tiếp tín hiệu GPIO 5 V nếu không xác minh và chuyển mức.

Theo Chapter 2: IoT Hardware, slide PDF 54–61.

2.20.1 ESP-01, ESP-07 và NodeMCU

Bảng 2.9: Những dạng phần cứng ESP8266 xuất hiện trong slide

Module / bo	Điểm cần nhớ
ESP-01	Module nhỏ (slide nêu khoảng 24.75×14.5 mm), anten PCB, chân hạn chế, nguồn 3.3 V; firmware AT phổ biến cho điều khiển qua UART.
ESP-07	Module 3.3 V; slide nêu anten gốm và đầu nối U.FL, Wi-Fi 2.4 GHz b/g/n, TCP/UDP; chú ý chân pitch nhỏ.
NodeMCU	Bo phát triển dùng module ESP8266, có USB-UART, bộ ổn áp và đưa chân ra hàng header; sơ đồ nhãn D0/D1... không đồng nhất số GPIO silicon.

2.20.2 Firmware, SDK và quy trình IDE

Slide đề cập Espressif SDK NonOS, RTOS SDK, ESP-Open-SDK và các hệ Arduino/ESP Easy/ESPHome. Đây là các lựa chọn có lịch sử phát triển riêng, không phải cùng một thư viện. Đối với ví dụ lịch sử trên slide, cài ESP8266 Arduino core bằng cách thêm Boards Manager URL rồi cài package ESP8266 và chọn Generic ESP8266 Module; *URL và phiên bản package có thể thay đổi*, phải xem tài liệu chính thức hiện hành.

Theo Chapter 2: IoT Hardware, slide PDF 58–61.

2.21 ESP32: một họ SoC, không phải một chip duy nhất

ESP32 được Espressif phát triển với nhiều bộ xử lý, ngoại vi và khả năng vô tuyến tùy *dòng chip*. Slide mô tả ESP32 đời đầu có hai lõi, khoảng 160–240 MHz, hỗ trợ Wi-Fi/Bluetooth, nhiều GPIO và công năng kết nối cao hơn ESP8266, chi phí cao hơn. Tuy nhiên, có ESP32-S/C/H và các biến thể *không có cùng số lõi, Bluetooth, Wi-Fi, chân ADC hoặc giao diện USB*; không được áp dụng nguyên bảng thông số của ESP32 đời đầu cho toàn bộ họ ESP32.

Theo Chapter 2: IoT Hardware, slide PDF 62–65.

2.21.1 Lợi thế và giới hạn thực dụng

- **Tích hợp:** MCU và phần giao tiếp vô tuyến trên một chip/module giúp tiết kiệm diện tích và thời gian phát triển.
- **Ngoại vi:** GPIO, SPI, UART, I^2C, PWM, ADC, timer, interrupt; số lượng và ma trận ánh xạ chân phụ thuộc mã chip.
- **Nguồn:** phát Wi-Fi/BLE có dòng đỉnh cao; cần tụ decoupling và bộ ổn áp có dự trữ dòng. Giảm công suất bằng sleep hoặc giảm tần suất truyền khi tác vụ cho phép.
- **Phần mềm:** Arduino core thuận tiện học tập; ESP-IDF thích hợp khai thác phần mềm hệ thống, RTOS và ngoại vi chuyên sâu.
- **Tương thích:** cần kiểm tra chân bootstrap, chân dành riêng Flash, chân input-only (với một số chip), miền ADC, mức logic, loại giao thức Bluetooth cụ thể.

Ghi nhớ

Nhận định trên slide "ESP32 ra đời do ESP8266 thiếu bảo mật": nội dung bài giảng có đề cập như vậy, nhưng không nên xem đây là lời giải thích lịch sử đầy đủ. ESP32 còn mở rộng năng lực xử lý, ngoại vi và tính năng nền tảng. Các cơ chế bảo mật thực tế phụ thuộc dòng chip, boot configuration, firmware và cách triển khai sản phẩm.

2.21.2 So sánh các lựa chọn MCU căn bản

Bảng 2.10: So sánh định tính khi chọn nền tảng; không thay datasheet

Đặc điểm	ATmega328P/Uno	ESP8266	ESP32 (họ chip)
Kết nối mặc định	Không có IP/Wi-Fi trên MCU	Wi-Fi 2.4 GHz	Tùy dòng: Wi-Fi và/hoặc Bluetooth; không đồng nhất
Hệ logic	Bo Uno cổ điển 5 V	Chủ yếu 3.3 V	Chủ yếu 3.3 V
Năng lực xử lý	8-bit, RAM rất nhỏ	MCU 32-bit	Tùy dòng, nhiều ngoại vi
Lập trình	Arduino IDE, AVR-GCC	Arduino core/SDK	Arduino core/ESP-IDF
Phù hợp	Điều khiển cơ bản, học GPIO	Nút IoT Wi-Fi giá thấp	Nút IoT đa cảm biến, kết nối, Edge nhỏ

2.22 Intel Edison: ví dụ nền tảng lai có tính lịch sử

Theo slide, module Intel Edison có kích thước xấp xỉ $35.5 \times 25 \times 3.9$ mm, SoC Atom "Tangier" sản xuất tiến trình 22 nm, hai lõi Silvermont 500 MHz, lõi Quark 100 MHz, RAM 1 GB, eMMC 4 GB, hỗ trợ Wi-Fi, Bluetooth 4, USB và đầu nối 70 chân. Các bo mở rộng được giới thiệu gồm *breakout board* mức I/O 1.8 V với USB OTG, cổng cấp nguồn, sạc và cầu UART; *Arduino expansion board* đưa thêm chân kiểu Arduino (slide liệt kê 20 digital, 6 analog, UART, I^2C, SPI/ICSP, microSD).

Theo Chapter 2: IoT Hardware, slide PDF 66–68.

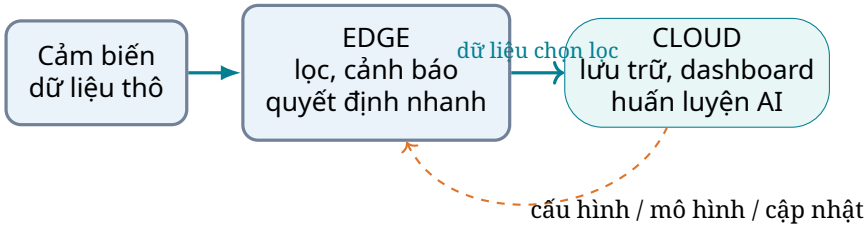
Ghi nhớ

Góc nhìn lịch sử: Intel Edison là nền tảng đã ngừng sản xuất/hỗ trợ. Chương này giữ để đọc hiểu slide và so sánh mô hình SoC + breakout + bo mở rộng, *không khuyến nghị* chọn làm nền tảng cho thiết kế sản phẩm mới. I/O 1.8 V cần đặc biệt chú ý khi nối module ngoài.

2.23 Edge Computing trong kiến trúc IoT

Điện toán biên (*edge computing*) chuyển một phần thu nhận, lọc, phân tích hoặc quyết định đến thiết bị gần nơi sinh dữ liệu hơn thay vì gửi toàn bộ dữ liệu thô lên đám mây. Bài giảng minh họa các tình huống *nhạy trễ*, *nhạy riêng tư*, bị hạn chế băng thông hoặc kết nối chập chờn; các thiết bị edge tiêu biểu có thể là MCU đủ mạnh, gateway Linux hoặc SBC.

Theo Chapter 2: IoT Hardware, slide PDF 69–72.



Hình 2.11: Sự phân công công việc điển hình giữa Edge và Cloud.

2.23.1 So sánh các vị trí xử lý

Bảng 2.11: Edge và Cloud: tiêu chí thiết kế

Tiêu chí	Xử lý Edge	Xử lý Cloud
Độ trễ	Có thể thấp, không nhất thiết phụ thuộc Internet	Có thêm trễ mạng và dịch vụ
Băng thông	Có thể lọc/gộp dữ liệu trước khi gửi	Có khả năng nhận tập dữ liệu lớn
Kết nối kém	Có thể tiếp tục một số chức năng	Khó dùng khi mất Internet
Tài nguyên	Giới hạn bởi công suất tính toán, pin, RAM	Co giãn dễ hơn, nhưng có phí/độ trễ
Riêng tư	Giảm dữ liệu thô rời thiết bị nếu thiết kế đúng	Cần chính sách truy cập và lưu trữ phù hợp

2.24 Raspberry Pi: máy tính đơn bo cho gateway và Edge

Raspberry Pi là họ máy tính đơn bo kích thước nhỏ, xuất hiện trong bài giảng như lựa chọn chạy hệ điều hành, kết nối mạng, làm gateway, hiển thị và phân tích dữ liệu. Khác với Uno, Raspberry Pi có thể chạy Linux và nhiều chương trình đồng thời, song khởi động chậm hơn, cần nguồn ổn định và bảo vệ lưu trữ. Slide đưa ví dụ Raspberry Pi 4 và những thành phần của kit: bo Pi, thẻ microSD, nguồn, bàn phím/chuột USB, màn hình HDMI, cáp HDMI, vỏ bảo vệ.

Theo Chapter 2: IoT Hardware, slide PDF 72–77.

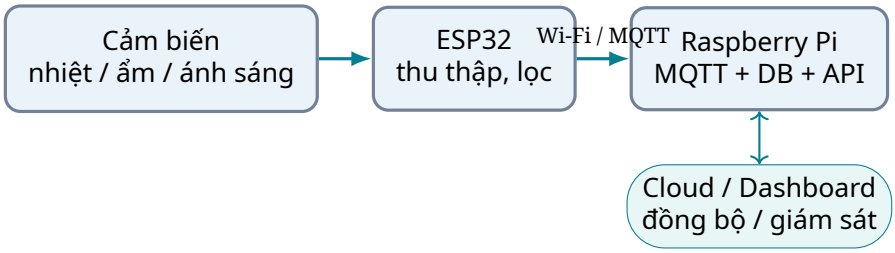
2.24.1 Các thành phần và GPIO

Raspberry Pi 4 minh họa trong slide có các khu vực: SoC, RAM, cổng USB, Ethernet, HDMI dạng micro, USB-C cấp nguồn, đầu camera/display và hàng chân mở rộng 40-pin. **GPIO của Raspberry Pi hoạt động theo logic 3.3 V**, không được đưa trực tiếp điện áp 5 V vào GPIO nếu không có mạch giao tiếp phù hợp. Số chân header vật lý, số GPIO và chức năng alternate không đồng nhất.

Cần phân biệt: USB-C trên Raspberry Pi 4 chủ yếu dùng cấp nguồn; điều đó không đồng nghĩa mọi Pi có thể lập trình/nạp trực tiếp như ESP32 qua cùng cổng. Với Pi chạy Linux, lập trình thường thông qua SSH, terminal, editor hoặc copy file lên hệ thống; môi trường và cách boot khác MCU.

2.24.2 Dựng gateway cơ bản với Raspberry Pi

Một kiến trúc ví dụ: cảm biến nối với ESP32, ESP32 truyền dữ liệu qua Wi-Fi/MQTT đến broker trên Raspberry Pi, Pi lưu cơ sở dữ liệu/đẩy lên dashboard/cloud. Nếu mạng Internet bên ngoài mất, dữ liệu có thể vẫn được lưu ở Pi trong LAN; khi có mạng lại, tiến hành đồng bộ theo chính sách ứng dụng.



Hình 2.12: Ví dụ nguyên lý mạng IoT tại nhà dùng ESP32 và Raspberry Pi.

2.24.3 Arduino và Raspberry Pi: không có nền tảng nào tốt nhất cho mọi việc

Bảng 2.12: Bảng kết luận đối chiếu nội dung cuối slide

Thuộc tính	Arduino Uno (ATmega328P)	Raspberry Pi 4 (SBC)
Bộ xử lý	MCU 8-bit điều khiển đơn giản	SoC mạnh chạy Linux
Hệ điều hành	Sketch chạy trên MCU, không có Linux	Hệ điều hành và đa tiến trình
Thời gian khởi động	Thường nhanh và đơn giản	Cần boot OS từ thẻ/lưu trữ
Tính xác định	Thuận lợi cho I/O định thời đơn giản	Linux thông thường không bảo đảm hard real-time
Mạng	Cần shield/module nếu muốn Wi-Fi	Có giao tiếp mạng tích hợp ở một số model
Ứng dụng	Cảm biến, đóng ngắt, điều khiển thời gian đơn giản	Gateway, database, dashboard, xử lý hình ảnh
Chi phí, điện năng	Thường thấp hơn ở lớp nút đơn giản	Thường cao hơn về điện năng/tài nguyên

Theo Chapter 2: IoT Hardware, slide PDF 78.

2.25 Thiết kế nút IoT: bài toán tổng hợp nhiều nền tảng

2.25.1 Bảng quyết định chọn phần cứng

Bảng 2.13: Checklist thiết kế từ yêu cầu tới linh kiện

Câu hỏi	Quyết định kỹ thuật tương ứng
Dữ liệu cần đo là gì?	Cảm biến, số trục, khoảng đo, accuracy, giao tiếp và tốc độ lấy mẫu
Tác động tới môi trường?	LED/servo/motor, driver, điện áp, dòng đỉnh và mạch bảo vệ
Làm việc khi mất mạng?	Đặt xử lý thiết yếu trên MCU/Edge; đệm dữ liệu và xác thực gửi lại
Tốc độ/khối lượng xử lý?	MCU 8-bit, ESP32, SBC Linux, bộ nhớ cần thiết
Cần Wi-Fi/BLE/RF?	SoC tích hợp hay module ngoài, băng thông, vùng phủ, anten
Nguồn pin hay nguồn cố định?	Dòng active/sleep/peak, nguồn ổn áp và sạc, thời lượng mục tiêu
Dung lượng dữ liệu?	RAM, Flash, EEPROM, microSD, độ bền ghi và khả năng phục hồi
Bảo mật?	Danh tính thiết bị, xác thực broker/API, TLS khả thi, firmware OTA an toàn
Độ ổn định vận hành?	Watchdog, reset, brownout, lọc nhiễu, xác minh wiring và logging
Bảo trì về sau?	Datasheet, sơ đồ chân, BOM, README, phiên bản firmware, test plan

2.25.2 Công thức sơ bộ về lưu trữ và truyền thông

Nếu một mẫu có B byte và tốc độ f_s mẫu/giây, trong thời gian T giây thì dữ liệu thuần tối thiểu:

$$\boxed{M = B\, f_s\, T} \quad \text{byte}, \tag{2.6}$$

chưa tính timestamp, JSON/CSV, header mạng, kiểm tra lỗi và hệ số dự phòng. Ví dụ mỗi mẫu 24 byte, 10 Hz trong 1 giờ: $24 \cdot 10 \cdot 3600 = 864{,}000$ byte (khoảng 0.864 MB hệ thập phân). Khi ghi microSD/Flash phải tính cả kích thước block và tuổi thọ ghi.

2.25.3 Ví dụ hệ IoT đầy đủ: giám sát phòng học

1. **Nhu cầu:** giám sát nhiệt độ/độ ẩm, độ sáng và thông báo nếu vượt ngưỡng; đồng thời quản lý quạt thông gió.
2. **Cảm biến:** chọn module nhiệt–ẩm số và cảm biến ánh sáng, đánh giá dải đo, sai số và điều kiện lắp.
3. **Bộ xử lý:** chọn ESP32 nếu cần Wi-Fi trực tiếp; chọn Uno kèm module Wi-Fi nếu cần học các khối tách rời; chọn Raspberry Pi làm gateway/dashboard.
4. **Actuator:** relay/MOSFET phù hợp điện áp, dòng tải và chuẩn an toàn; thiết lập trạng thái fail-safe nếu MCU lỗi.

5. **Phần mềm:** ước lượng chu kỳ lấy mẫu, chuẩn hóa dữ liệu, timestamp, MQTT hoặc HTTP, cấu hình ngưỡng và xử lý mất kết nối.
6. **Kiểm thử:** chạy đo trong điều kiện bình thường, biên cao/thấp, rút mạng, cấp nguồn lại, thử dòng đỉnh của tải và log lỗi.

2.26 Câu hỏi tự luyện và đáp án ngắn

1. **MCU khác SBC ở đâu?** – MCU tích hợp CPU, RAM, Flash và ngoại vi nhúng; SBC là hệ máy tính hoàn chỉnh thường chạy OS và có nhiều tài nguyên hơn.
2. **ATmega328P có mấy loại bộ nhớ chính theo slide?** – Flash 32 KB, SRAM 2 KB, EEPROM 1 KB.
3. **Arduino Uno cổ điển có bao nhiêu ngõ digital và analog?** – 14 digital và 6 analog được đưa ra chân trên bo, không đồng nghĩa chip chỉ có từng đó chức năng.
4. **Vai trò `setup` và `loop`?** – Cấu hình một lần; vòng xử lý lặp liên tục.
5. **Serial port chân nào trên Uno?** – D0/RX và D1/TX; GND và mức điện áp phải tương thích.
6. **ESP-01 khác NodeMCU?** – ESP-01 là module nhỏ với ít chân; NodeMCU là bo phát triển tích hợp USB-UART/nguồn, tiện lập trình.
7. **ESP32 có luôn dual-core và Bluetooth Classic?** – Không; đặc tính tùy series/model.
8. **Tại sao Intel Edison xuất hiện trong bài?** – Ví dụ lịch sử về module SoC + breakout/Arduino board; không dùng làm lựa chọn hiện đại mặc định.
9. **Khi nào cần Edge?** – Khi ưu tiên trễ thấp, giảm băng thông, bảo vệ riêng tư và duy trì tác vụ tại chỗ khi mất mạng.
10. **Raspberry Pi khác Arduino về môi trường chạy?** – Pi thường chạy Linux; Uno chạy sketch nhúng, không có hệ điều hành đa nhiệm tổng quát như Linux.

2.27 Bảng thuật ngữ ôn thi

Thuật ngữ	Diễn giải vắn tắt
MCU	Microcontroller Unit – vi điều khiển đơn chip tích hợp CPU, bộ nhớ, ngoại vi.
SoC	System-on-Chip – hệ thống tích hợp trên chip, thường có CPU, ngoại vi và khối chuyên dụng.
SBC	Single-board Computer – máy tính trên một bo, thường chạy OS.
GPIO	General-Purpose Input/Output – chân nhập/xuất số đa dụng.
ADC	Analog-to-Digital Converter – bộ chuyển tương tự sang số.
PWM	Pulse-Width Modulation – điều chế độ rộng xung.
UART	Universal Asynchronous Receiver/Transmitter – nối tiếp bất đồng bộ.
SPI	Serial Peripheral Interface – giao tiếp nối tiếp đồng bộ, thường có CS.
I^2C	Bus SDA/SCL hai dây, nhiều thiết bị theo địa chỉ.

Thuật ngữ	Diễn giải vắn tắt
Bootloader	Phần mềm khởi động hỗ trợ nạp firmware qua giao diện quy định.
OTA	Over-the-Air – cập nhật firmware qua mạng vô tuyến.
Shield	Bo cắm mở rộng cho một nền tảng (ví dụ Arduino).
Breakout board	Bo đưa các chân/tính năng của module/chip ra chân kết nối dễ dùng.
Edge Computing	Xử lý ở gần nơi sinh dữ liệu thay vì chuyển hết về Cloud.
MQTT	Giao thức publish/subscribe nhẹ phổ biến trong IoT, sử dụng broker.

Ghi nhớ

Tóm tắt cần nhớ: Phân biệt *MCU – module – board – SBC*. ATmega328P/Arduino thích hợp nền tảng học điều khiển; ESP8266/ESP32 bổ sung kết nối không dây và năng lực xử lý; Intel Edison là tư liệu lịch sử; Raspberry Pi hợp lý cho gateway và ứng dụng Linux. Edge giúp giảm độ trễ và phụ thuộc Cloud. Luôn đối chiếu sơ đồ chân, điện áp logic và nguồn trước khi lắp mạch.